Unit 2 Assignment: Linear Data Structures (Arrays, Lists, Stacks, Queues)

Name: Vedansh Rawat

Roll no : 2501730362

Btech Cse(AI & ML) Section F

Task-1 Arrays Vs Linked Lists

Aspect	Arrays	Linked Lists
Memory	Contiguous memory	Non-contiguous memory
Access Time	O(1) random access (direct indexing)	O(n) sequential access (must traverse from head)
Memory Overhead	Low (just data, but may have unused capacity)	Higher (extra memory for pointers in each node)
Insertion/Deletion (Beginning)	O(n) (shift all elements)	O(1) (just update head pointer)
Insertion/Deletion (End)	O(1)	O(n)
Real-World Use Cases	1. Storing fixed-size datasets 2. Fast lookup systems	1. Implementing stacks/queues 2. Dynamic memory allocation

Insertion at Index 0: Array vs Linked List

Array:
Arrays store elements in contiguous memory. To insert at index 0, all existing elements are shifted one position to the right to make space. Then the new element is placed at the first position.
Time Complexity: O(n)

Linked List:
Linked lists store elements in non-contiguous memory using pointers. To insert at index 0, a new node is created, its pointer is set to the current head, and the

head is updated to the new node. No shifting is required.
Time Complexity: O(1)

Choose the best DS and justify in 3–5 lines

(a) Storing fixed-size marks of 60 students

Data Structure: Array
Arrays allow direct index-based access (O(1)), which is ideal when using roll numbers.
Since the size is fixed (60 students), no resizing is needed.
It provides fast and efficient retrieval of marks.

(b) Music playlist (frequent insert/remove in middle)

Data Structure: Linked List
Linked lists allow easy insertion and deletion without shifting elements.
This is useful when modifying the playlist frequently.
Only pointer updates are required, making operations efficient.

(c) Browser "Back" history feature

Data Structure: Stack
A stack follows LIFO (Last In, First Out) order.
The most recently visited page is accessed first when pressing "Back."
Push and pop operations efficiently manage browsing history.

(d) Print queue in a lab

Data Structure: Queue
A queue follows FIFO (First In, First Out) order.
Print jobs are processed in the order they are received.
Ensures fair and sequential handling of tasks.

Task 2: Numericals on Array Address Calculation

An integer array A starts at base address BA = 2000. Each integer takes w = 4 bytes. Compute the address of A[0], A[5], A[17] using: LOC(A[i]) = BA + (i × w)

Base Address (BA) = 2000
Element size (w) = 4 bytes
Formula: LOC(A[i]) = BA + i × w

- LOC(A[0]) = 2000 + (0 × 4) = 2000
- LOC(A[5]) = 2000 + (5 × 4) = 2000 + 20 = 2020

- LOC(A[17]) = 2000 + (17 × 4) = 2000 + 68 = 2068

A 2D array B has size R × C = 4 × 6, base address BA = 5000, element size w = 2 bytes. Compute the address of: B[0][0], B[2][3], B[3][5] using: LOC(B[i][j]) = BA + (i × C + j) × w Also mention what happens if someone asks B[4][0] (out of range)

R = 4, C = 6
Base Address (BA) = 5000
Element size (w) = 2 bytes
Formula: LOC(B[i][j]) = BA + (i × C + j) × w

- LOC(B[0][0]) = 5000 + (0×6 + 0)×2 = 5000
- LOC(B[2][3]) = 5000 + (2×6 + 3)×2 = 5000 + (12 + 3)×2 = 5000 + 30 = 5030
- LOC(B[3][5]) = 5000 + (3×6 + 5)×2 = 5000 + (18 + 5)×2 = 5000 + 46 = 5046

Accessing B[4][0] is out of bounds and leads to undefined behavior (may cause error or access invalid memory).

Why does direct indexing in arrays feel "instant" (fast), but in a linked list it is slow to reach the k-th node?

Direct indexing in arrays feels "instant" because arrays store elements in contiguous memory, so the address of any element can be calculated directly using a formula based on its index. This allows access in constant time, O(1), without depending on the array size. In contrast, a linked list stores elements in non-contiguous memory, where each node points to the next. To reach the k-th node, you must start from the head and follow pointers one by one, making the process sequential and O(n), which is slower.

Task 3: Dynamic Array Simulation

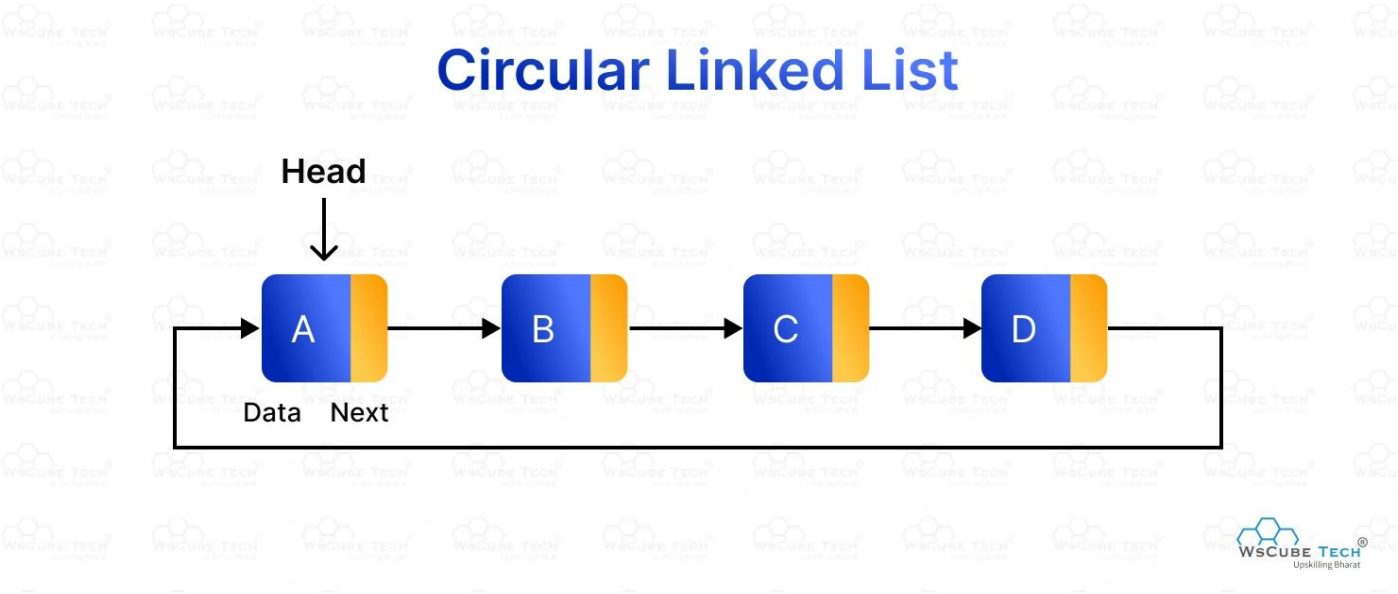
Explain why append(x) is amortized O(1) even though resizing sometimes costs O(n).

In a dynamic array, most append(x) operations take O(1) time because the element is simply added at the next free space. However, when the array becomes full, it needs to be resized (usually doubled), and all existing elements are copied to a new array, which takes O(n) time.

This expensive resizing does not happen often—it occurs rarely compared to the many cheap append operations. Over a large number of appends, the total cost of occasional expensive operations gets spread out (amortized) over all operations.

Task 4: Linked Lists

Explain how a Circular Linked List differs. Also mention one real-world situation where circular behavior is useful.



A circular linked list is a type of linked list in which the last node points back to the first node (head) instead of pointing to NULL. This forms a loop, allowing traversal to continue indefinitely.

Here, the last node (D) points back to the first node (A), creating a cycle.

How it differs from a normal linked list:

- In a singly linked list, the last node points to NULL.
- In a circular linked list, the last node points to the head, forming a loop.
- Traversal in a circular list can continue endlessly, while in a normal list it stops at NULL.

Real-World Use Case:

Round-robin scheduling (CPU scheduling)
Processes are arranged in a circular list, and each process gets a fixed time slice. After the last process, the scheduler returns to the first, ensuring fair and continuous execution.

Task 5: Stack & Queue

5.3 Complexity Table (Array vs Linked List)

Operation	Array	Reason	Linked-Lists	Reason
Push (Stack)	O(1)	Direct insert at end	O(1)	Insert at head, no shifting
Pop (Stack)	O(1)	Remove last element directly	O(1)	Remove from head
Peek (Stack)	O(1)	Direct index access	O(1)	Access head node
Enqueue (Queue)	O(1)	Insert at end	O(1)	Insert at tail
Dequeue (Queue)	O(n)	Removing front requires shifting elements	O(1)	Remove from head
Front (Queue)	O(1)	First element directly accessible	O(1)	Access head node

Task 6: Application – Parentheses Checker

Manually demonstrate the stack states for:

(i) Expression: "([])" → Balanced

Step	Symbol	Stack (Top → Right)
1	(	(
2	[	([
3	]	(*(pop [)*
4	)	*(pop ()* → empty

Final Stack: Empty → Balanced

(ii) Expression: "([)]" → Not Balanced

Step	Symbol	Stack (Top → Right)
1	(	(
2	[	([
3	)	Mismatch (top is [, expected ()

Result: Not Balanced (invalid immediately)